

JAVA FULLSTACK CURRICULUM



HTML5 (Structure & Semantics)

- Doctype, Headings, paragraphs, text formatting
- Links & images
- Lists (ordered, unordered, definition)
- Tables (with colspan/rowspan)
- Forms & inputs (basic + HTML5 types like email, number, date)
- Semantic elements (header, nav, section, article, footer, aside)
- Multimedia (video, audio, iframe)

Practice: Build a multi-page portfolio structure with HTML only.

CSS (Styling & Layouts)

- CSS basics: selectors, properties, colors, fonts
- Box Model (margin, padding, border)
- Display: block, inline, inline-block, none
- Positioning: relative, absolute, fixed, sticky
- Pseudo-classes & pseudo-elements (:hover, ::before)

Practice: Style your HTML portfolio make it responsive.

Flexbox & CSS Grid

Flexbox

- display:flex, direction, wrap
- Justify & align content
- Flex grow/shrink/basis

CSS Grid

- display:grid, rows & columns
- Grid areas, template layouts
- Combining grid & flexbox

Practice: Build a responsive landing page with Flexbox & Grid.

Bootstrap (Rapid UI Framework)

- Why use Bootstrap vs custom CSS
- Setup via CDN & file structure
- Grid system (12 columns, breakpoints)
- Utilities (spacing, colors, display)
- Components: navbars, buttons, forms, cards, modals

Practice: Create a responsive blog or e-commerce product grid

Web Design Principles (UI/UX)

- Color theory & accessibility contrast ratios
- Typography best practices (Google Fonts)
- Layout principles (proximity, alignment, white space)
- Wireframing basics
- UX heuristics (consistency, feedback, error prevention)

Practice: Redesign your portfolio with UI/UX principles.

JavaScript (ES6 & DOM)

Basics

- What is JS? Role in browsers
- Variables (let, const, var)
- Data types & operators
- Strings, numbers, arrays, objects
- Functions (declarations, expressions, arrow functions, higher-order functions)
- Loops & conditionals

DOM & Events

- DOM tree, querySelector, getElementById
- Changing HTML/CSS dynamically
- Event handling (addEventListener, event bubbling, delegation)
- Form validation & regex
- LocalStorage & SessionStorage

Advanced JS

- Array methods (map, filter, reduce)
- OOP (classes, constructors, this, inheritance) JSON handling
- Fetch API & async/await
- Error handling (try...catch)
- Modular JS (import/export)
- Common design patterns (Module, Observer)

Practice Projects:

- To-do list with LocalStorage
- Calculator app
- Weather app using Fetch API
- Interactive quiz app

Angular (Frontend Framework)

Angular Basics

- Angular setup (CLI, project structure)
- Components, Modules, Templates
- Data Binding (Interpolation, Property, Event, Two-way)
- Directives (ngIf, ngFor, ngSwitch)

Mini Project: Task Tracker App

Angular Forms & Services

- Template-driven vs Reactive forms
- Form validation (sync & async)
- Dependency Injection
- Angular Services & HttpClient (API calls)

Mini Project: Weather Dashboard with API Integration

Angular Routing + Full-Stack Integration

- Angular Router (navigation, route params, guards)
- Lazy loading modules
- Consuming REST APIs (Spring Boot / Node backend)
- Displaying data in Angular (tables, lists, cards, pagination)

Mini Project: Frontend for Product Catalog

Angular + Authentication

- JWT Authentication flow (login, signup, logout)
- Storing tokens (LocalStorage, Http Interceptors)
- Role-based route guards (Admin/User access)

Mini Project: Secure User Login + Dashboard

Java Backend Development Curriculum

1. Core Java (Foundations)

Goal : Build strong fundamentals in Java programming.

- Java Basics
- JVM, JRE, JDK
- Data types, variables, operators
- Control structures (if, switch, loops)
- Object-Oriented Programming (OOP)
- Classes, Objects, Constructors
- Inheritance, Polymorphism, Abstraction, Encapsulation
- Core APIs
- String, StringBuilder, Wrapper classes
- Collections Framework (List, Set, Map, Queue)
- Generics
- Exception Handling
- Input/Output (I/O Streams, File handling)
- Multithreading & Concurrency (Thread, Runnable, Executors) - Java 8+ Features
- Lambda expressions, Streams API, Functional interface

2. Databases & SQL

Goal : Work with relational databases efficiently

- Relational Database Concepts
- SQL Basics
- SELECT, INSERT, UPDATE, DELETE
- WHERE, GROUP BY, HAVING, ORDER BY
- Joins (INNER, LEFT, RIGHT, FULL)
- Constraints & Indexes

3. Spring Framework & Spring Boot

Goal : Build production-grade Java applications easily

- Spring Core
- IoC Container, Beans, Dependency Injection
- Spring MVC
- Controllers, Models, Views
- Spring Boot
- Auto-configuration, Starter dependencies
- RESTful APIs (GET, POST, PUT, DELETE)
- Request/Response handling with JSON
- Exception handling
- Application properties & Profiles
- Spring Data JPA & Hibernate
- ORM basics
- Entity mapping (OneToOne, OneToMany, ManyToMany)
- Query Methods, JPQL, Native Queries
- Spring Security (Basic + JWT Authentication)
- Validation & AOP (Aspect Oriented Programming)

Scan to Start Your IT Career at JZ

